

第7章 开源WebGIS开发技术

第一节 开源软件概述

第二节 OpenLayers

第三节 GeoServer

第一节 开源软件概述

1. 开源软件

(1) 定义

开源软件是软件著作权人基于自由、开放和共享的理念，通过**开源软件许可证**的方式授予软件使用者在遵守许可限制条件下，可**自由使用、复制、改编、再发布的软件**。

(2) 一些著名开源许可类型

- **BSD许可** -- 它要求版权和著者身份申明；
- **GPL** -- 通用公共许可(“CopyLeft”)；
- **LGPL** -- 库/次级GPL。在这个许可证下，库（函数库）可以自由地联接到专有软件；
- **Artistic许可** -- 使作者保持对进一步开发的控制；
- **NPL许可** -- 基于GPL精神的新许可证，但是保持了对源码更多的控制和所有权；
- **公共域（Public Domain）** -- 通常用于决定性的算法以鼓励广泛使用，通常由美国政府使用；

! 开源并不等于完全自由使用，有许多不同的开源[软件许可](#)。

i> 使用开源代码的程序必须公开源代码

这类开源软件许可证最为严苛，要求任何使用该开源代码的程序在再发布时，必须将其源代码以相同的许可证公开，典型代表是 **GPL** 许可证的三个版本，包括：**GPLv1** 许可证、**GPLv2** 许可证和 **GPLv3** 许可证。

GPL许可证：General Public License（GPL）

通用公共许可证

是自由软件联盟（**GNU**）的开放源码软件许可证的一种，是开放源码软件领域最富盛名的一种许可证。它集中体现了纯粹的**Copyleft**精神。

Copyleft规则：是一种全新的版权许可模式，目的在于促进和保障自由软件的传播和发展，其内涵在于在承认传统版权的前提下，将复制权、发行权、修改权等权利通过GPL许可证的方式释放给社会公众，任何从copyleft许可衍生出的软件作品同样必须遵守copyleft许可的规定。

自由软件授权模式(copyleft)与传统版权(copyright);

ii> 使用并且修改开源代码的程序必须公开源代码

这类开源软件许可证与上述 **GPL** 开源软件许可证相比较为宽松，规定使用并且修改了该开源代码的程序在再发布时，必须将其源代码以相同的许可证公开，但新增代码不需要采用同样的许可证发布。

LGPL许可证：Lesser General Public License

iii>使用或修改开源代码的程序不强制公开源代码

BSD许可证： Berkeley Software Distribution license 是自由软件中使用最广泛的许可证之一。

BSD许可证具备开源软件许可证普遍的要求，但它只要求被许可者附上该许可证的原文以及所有开发者的版权资料。通俗地说，BSD许可证看重的是“名”，在“利”方面，BSD许可证给予被许可者充分使用（包含商业使用）源代码的权利。

BSD许可证完全不会产生像GPL许可证那样的“病毒效应”，为开源软件，特别是作为学校、公共科研机构科研成果的开源软件转化为产品等充分利用敞开了大门。

第二节 OpenLayers

一、OpenLayers概述

二、OpenLayers体系结构

三、OpenLayers工作原理

四、开发初步

一、OpenLayers 概述

企业级WebGIS前端渲染引擎

1.简介:

- **OpenLayers**作为业内使用最为广泛的地图引擎之一，已被各大GIS厂商和广大WebGIS开发者采用。借助于OpenLayers 强大的扩展功能，可以与各大WebGIS厂商的平台产品（如ArcGIS Server、MapGIS IGServer、SuperMap iServer、GeoServer等）相结合，开发出各具特色的WebGIS应用系统；
- OpenLayers 是一个专为Web GIS 客户端开发提供的 **JavaScript类库包**，通过**BSD License**发行；用于实现标准格式发布的地图数据访问；
- 除了支持网页端，还支持移动端；



OpenLayers makes it easy to put a dynamic map in any web page. It can display map tiles, vector data and markers loaded from any source. OpenLayers has been developed to further the use of geographic information of all kinds. It is completely free, Open Source JavaScript, released under the 2-clause BSD License (also known as the FreeBSD).

<http://openLayers.org>

2.OpenLayers的特征

- 支持瓦片图层
- 支持矢量切片
- 支持矢量图层
- 支持OGC规范
- 运用前沿技术
- 面向对象的思想
- 易于定制和扩展
- 优秀的交互体验

3. 应用案例



采用的技术

- 前端: **OpenLayers**
- 地图服务器:
GeoServer
- 空间数据库:
PostGIS
- 影像处理: **GDAL**

二、体系结构

OpenLayers的目的就是为了能够在客户端更好地展现和操作地图。**OpenLayers** 将抽象事物具体化为类，其核心类是**Map**、**Layer**、**Source**、**View**。几乎所有的动作都围绕这些核心类展开，从而实现地图加载和相关操作。

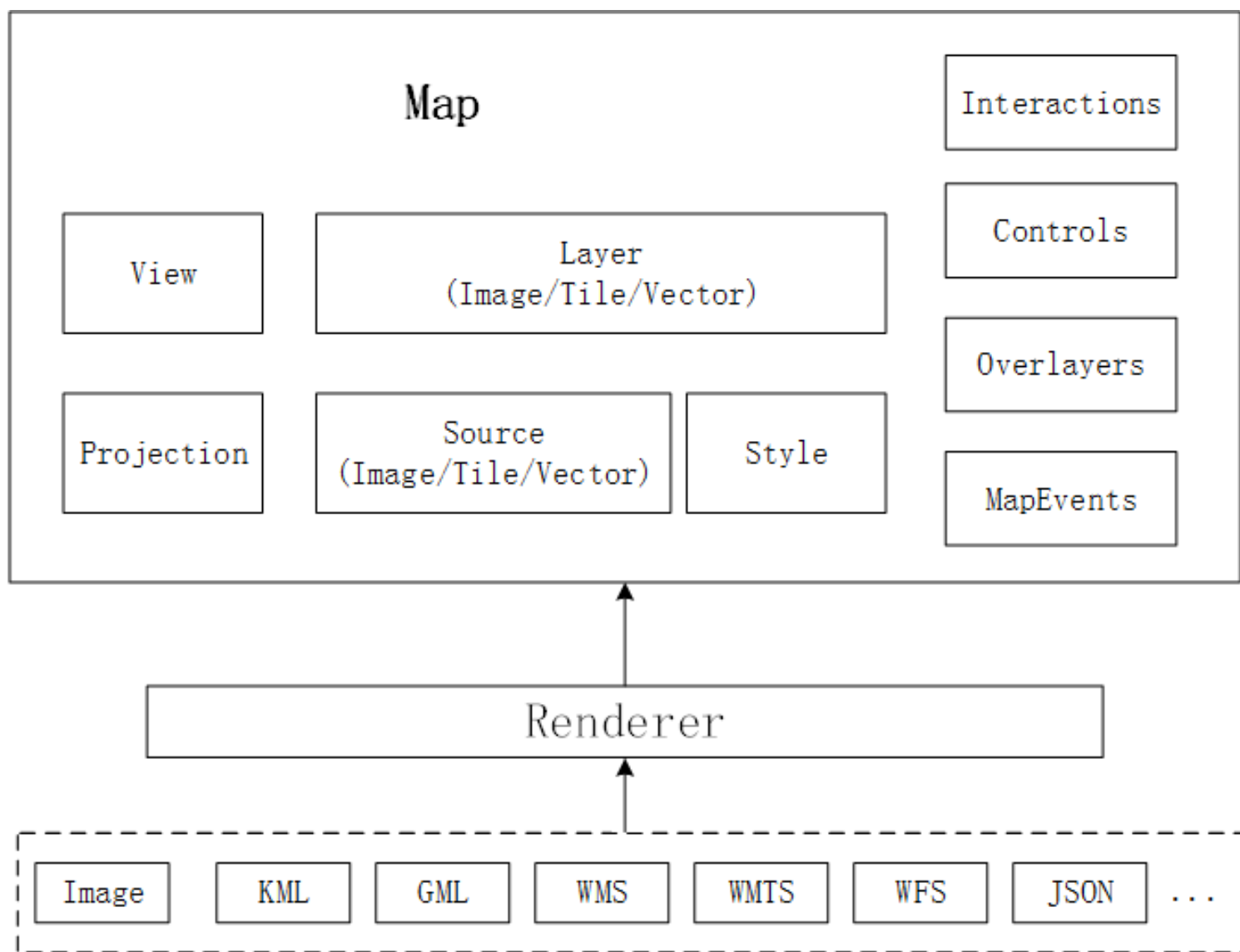


图 OpenLayers 5 体系结构框架

三、工作原理

针对复杂的空间数据：

- **Openlayers**如何将这些空间数据抽象为类？
- 如何解析各种空间数据源？
- 又是如何渲染并在客户端展示？

1.数据组织

(1) OpenLayers 5的数据源

地图数据根据数据源(Source)可分为**Image**、**Tile**、**Vector**三大类，对应设置到Image、Tile、Vector三大类图层(Layer)中。其中，矢量图层Vector通过样式(Style)来设置矢量数据渲染的方式和外观。

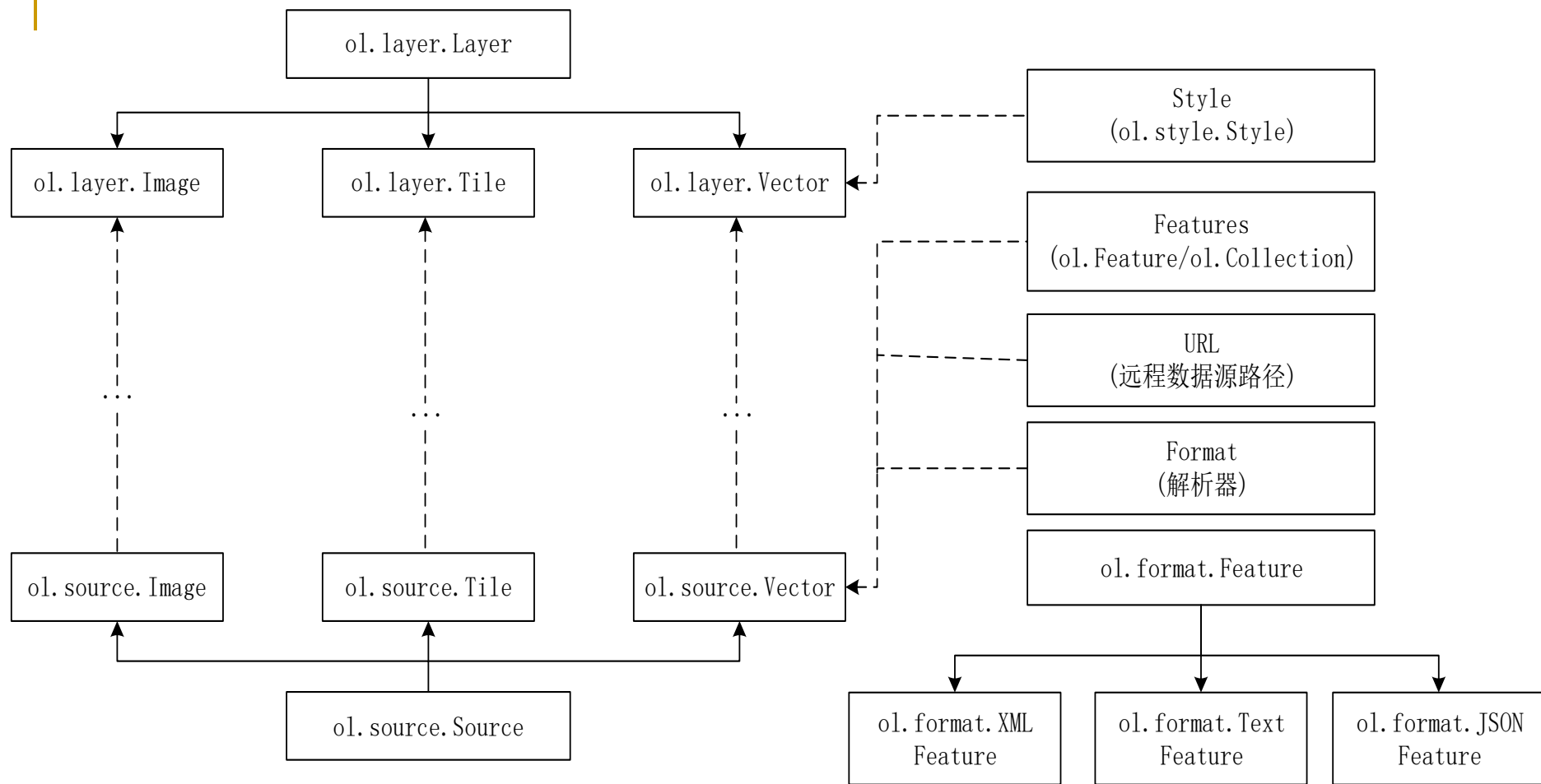


图 地图图层与数据源的关系

- **Image类**为单一图像基类，其子类作为画布(canvas)元素、服务器图片、单个静态图片、WMS单一图像等的数据来源；
- **Tile类**为瓦片抽象基类，其子类作为各类瓦片数据的数据来源；
- **Vector类**则为矢量基类，可直接实例化创建矢量数据的数据源(支持各种格式的矢量数据)，其子类则为扩展的某类矢量数据的数据源；

- **ol.souce.Vector** 是矢量数据源基类，为矢量图层提供具体的数据来源，包括直接组织的矢量数据(**Featrues**)、远程数据源的矢量数据(即通过 **URL** 设置数据源路径)等；
- **URL**设置的矢量数据源，则通过解析器 **Format**(即 **ol.format.Feature**的子类)来解析**XML**、**Text**、**JSON**格式的各类矢量数据，如**GML**、**KML**、**GPS**、**WFS**、**WKT**、**GeoJSON**等地图数据。

(2) 矢量数据

从表现形态来看，空间数据的矢量数据由点、线、面三类要素构成，将这些要素对应到 **Web** 客户端表现时，需要抽象为相应的类，包括要素之间的关系。在 **OpenLayers 5** 中，矢量数据的抽象主要由 **ol.geom.Geometry** 类下的几何对象子类实现。

ol.geom.Geometry

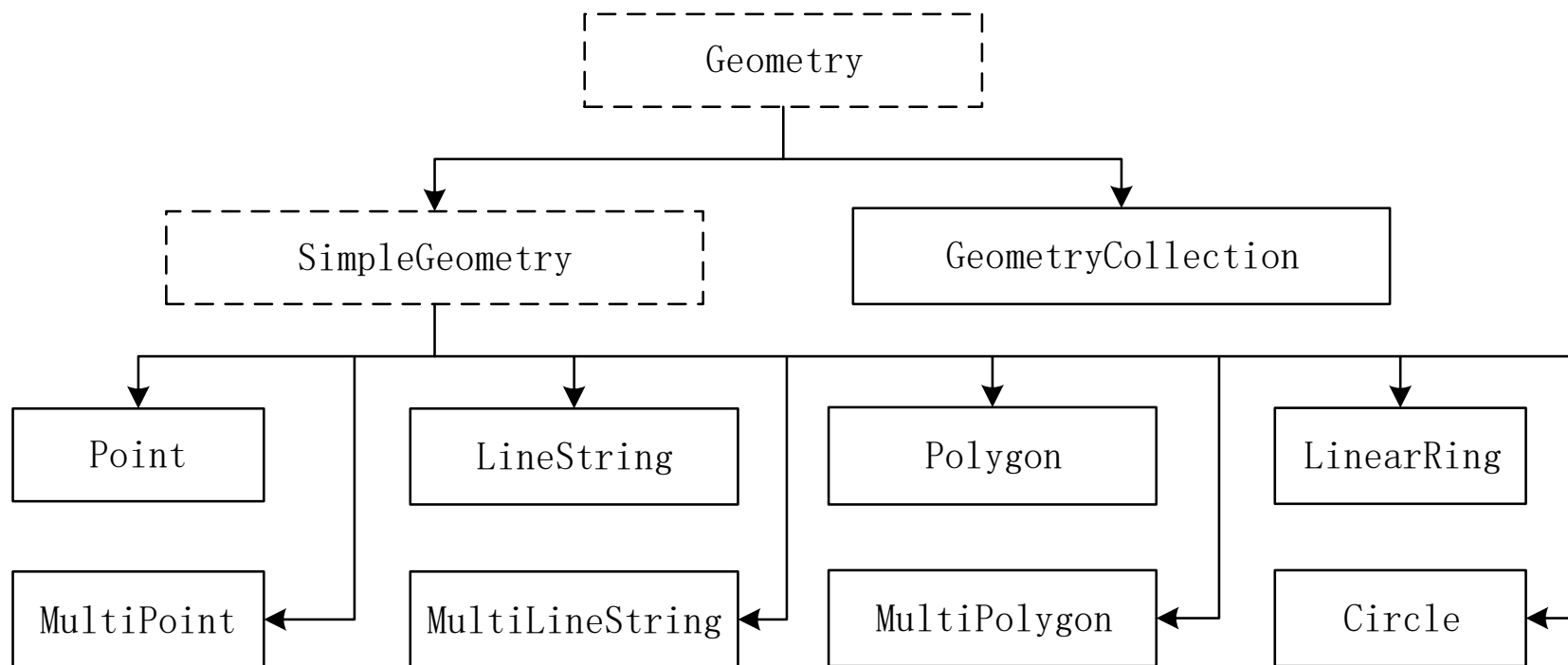


图 `Geometry`基类及其子类的继承关系

2.数据解析

Vector为矢量基类，由**Format**属性设置解析数据类型，即通过**ol.format.Feature**子类进行各种格式矢量数据的解析；由OpenLayers 5的API可知，**ol.format.Feature**类用于读/写各种格式的数据，并创建了各种格式的子类，即各类数据解析器，包括XML、Text、JSON类型的各种格式数据解析器。

数据解析（续）：

以GML数据解析为例，其实现原理为：

先通过接口调用得到GML格式的文本数据，然后通过ol.format.GML类的读/写方法来解析这个文本数据，从而读取矢量要素(Feature)及其几何对象(Geometry)等，最后结合样式(Style)并通过相应的渲染器在客户端渲染出来。

不管是什么格式的数据，最后解析得到基本的Point、LineString子类的Geometry对象，进行客户端渲染后就是在客户端看到的地图内容。

3.地图表现

- OpenLayers 5的核心是**Map类**、**Layer类**与**Source类**，结合**地图视图类View**、**矢量样式类 Style**等渲染地图。Map类的实例就是内嵌于 Web网页的交互式地图；
- 地图表现：简单说就是向一个地图容器里面加载图层与控件。Map类就是地图容器。可以分门别类地按空间层次来存放内容；
- 在Web网页中，地图容器通过div层来表现，将layers(含对应数据源source)、controls等内容加载到Map中，表现为地图。

地图表现（续）

- 对地图的渲染，必须由一个或多个图层类（Layers）、一个地图视图类（View）及一个目标容器（div）实现。
地图显示必备的三要素：
- View类主要控制用户与地图的基本交互，显示中心点、地图显示级别、范围等基本动作与内容。

四、开发初步

1. 获取OpenLayers5开发库

<https://github.com/openlayers/openlayers/releases/tag/v5.3.0>

- 开发库（开发与调试的js库、css样式）；
- 开发库资源包：开发库、示例、API；

- 开发库（开发与调试的js库、css样式）；

ol.js：核心开发库，集成当前OpenLayers5的所有功能。其中**ol.js.map**为**ol.js**的映射文件，调试用；

ol.css：样式类，包含OpenLayers5的所有默认样式信息。其中**ol.css.map**为**ol.css**的映射文件，调试用；

- 示例

- 使用API:

2.实例

(1) 用Open Layers实现地图联动功能。

```
<p>地图1</p>
<div id="map1" style="width: 100%"></div>
<p>地图2</p>
<div id="map2" style="width: 100%"></div>
<script>
  // 创建一个视图
  var view = new ol.View({
    center: [0, 0],
    zoom: 2
  });

  // 创建第一个地图
  new ol.Map({
    layers: [
      new ol.layer.Tile({source: new ol.source.OSM()})
    ],
    view: view,
    target: 'map1'
  });

  // 创建第二个地图
  new ol.Map({
    layers: [
      new ol.layer.Tile({source: new ol.source.OSM()})
    ],
    view: view,
    target: 'map2'
  });
</script>
```

地图组成部分：

- 地图(Map)，对应的类是`ol.Map`，地图文档；
- 视图(View)，对应的类是`ol.View`，控制地图显示的中心位置、范围、层级等；
- 图层(Layer)，对应的类是`ol.layer`，多个layer重叠在一起，相互之间互不干扰，有多种图层类型，对应多种图层数据源；
- 数据源(Source)，对应的类是`ol.source`，它是和图层一一对应的；支持多种多样在线或离线的数据源；可以是静态图或者瓦片图；也可以是栅格化的或者矢量的；
- 控件(Control)，对应的类是`ol.control`；
- 交互(Interaction)，对应的类是`ol.interaction`，用鼠标控制地图放大、缩小、移动，制作高级功能，如选择要素、添加点线面等；

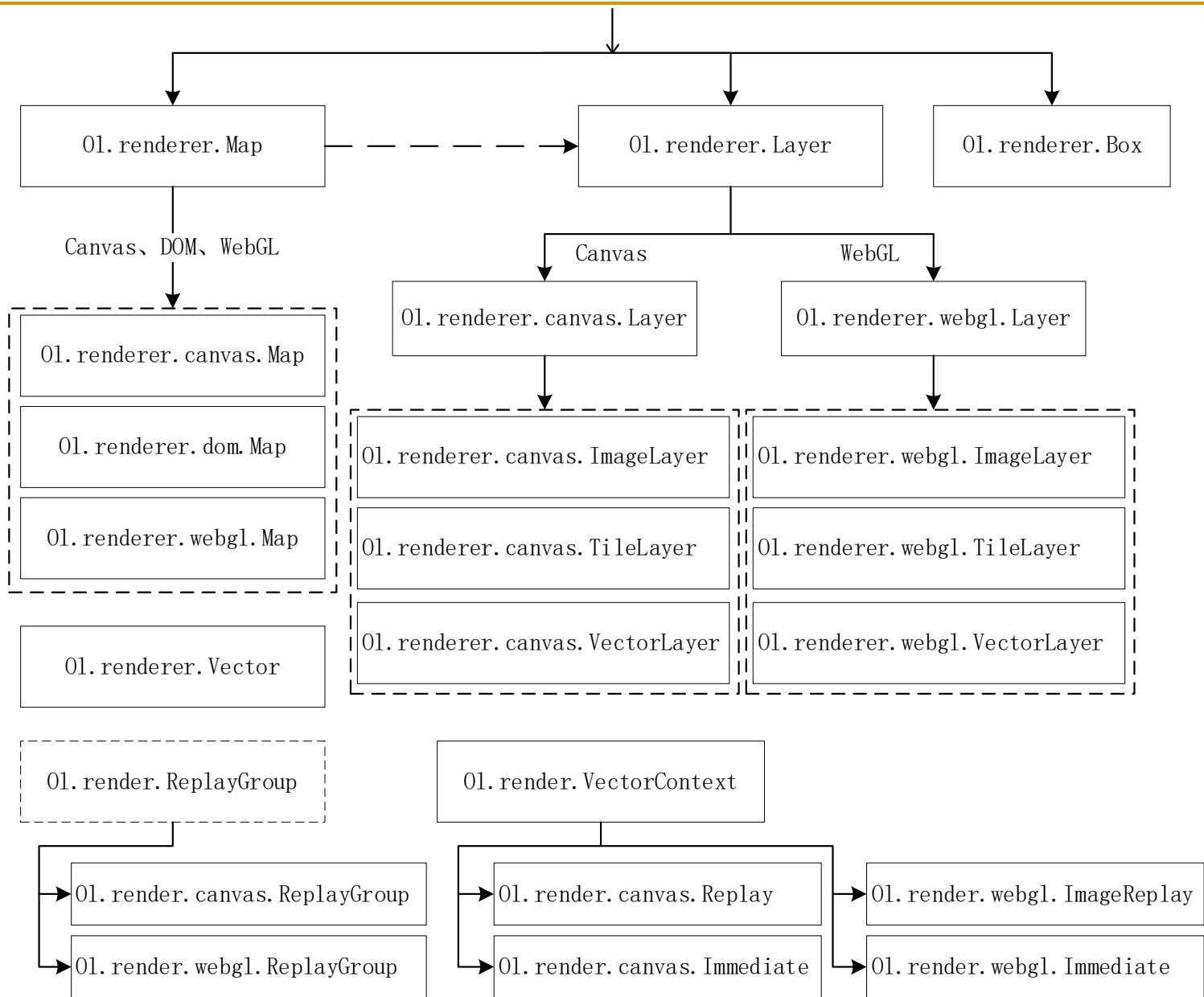
数据渲染：

OpenLayers 5的地图数据通过**图层 (Layer)**组织渲染，且通过**数据源(Source)**设置具体的地图数据来源。

基于OpenLayers的地图整个表现过程为：先通过URL**调用数据**，然后用各种解析器**解析数据**，再用相应的渲染器在**图层中进行渲染**，然后结合相应的控件表现出来，成为一幅“**动态**”**地图**。

在OpenLayers 5中，渲染功能有渲染器（即Render相关类）实现。**根据渲染方式（Canvas\WebGL）、与图层类型（Image、Tile、Vector）匹配渲染器将图层数据渲染出来。**

- **矢量数据的渲染机制：** 矢量数据的渲染支持**Canvas**、**WebGL**两种方式，其中*WebGL*渲染方式对矢量数据的支持有限且不支持矢量瓦片进行。矢量数据的渲染同时会结合样式（**style**）进行；
- 图像或瓦片数据在Web客户端仅仅是图片，不包含其他的几何信息和属性信息，均通过HTML中的**Canvas** 标签显示，即通过数据源对应的渲染器处理渲染；
- *OpenLayers*绘图的核心是通过**Canvas**进行绘制。



Render相关类关系

OpenLayers渲染:

- **OpenLayers中添加矢量数据需要经过三个步骤：创建数据源、创建矢量图层并设置样式、添加图层；**
- OpenLayers 中通过`ol.source.Vector`创建数据源，其参数`features`是多个要素形成的集合。在OpenLayers中，每个矢量图形都是一个通过`ol.Feature`创建的要素，每个要素包含`Geometry`和属性信息。`Geometry`的创建可以使用`ol.geom.Point`、`ol.geom.LineString`、`ol.geom.Polygon`方法，分别用于创建点对象、线对象和面对象。

示例：

线数据： `const coords = [[87.9947,39.491], [99.4908, 32.6796]]`

a.使用上述的线数据coords**创建一个矢量数据源：**

// 创建 geometry

`const geometry = new ol.geom.LineString(coords)`

// 创建要素集合 features

`const features = [new ol.Feature(geometry)]`

// 创建数据源 source

`const source1 = new ol.source.Vector({
features: features,
})`

b.创建矢量图层并设置样式

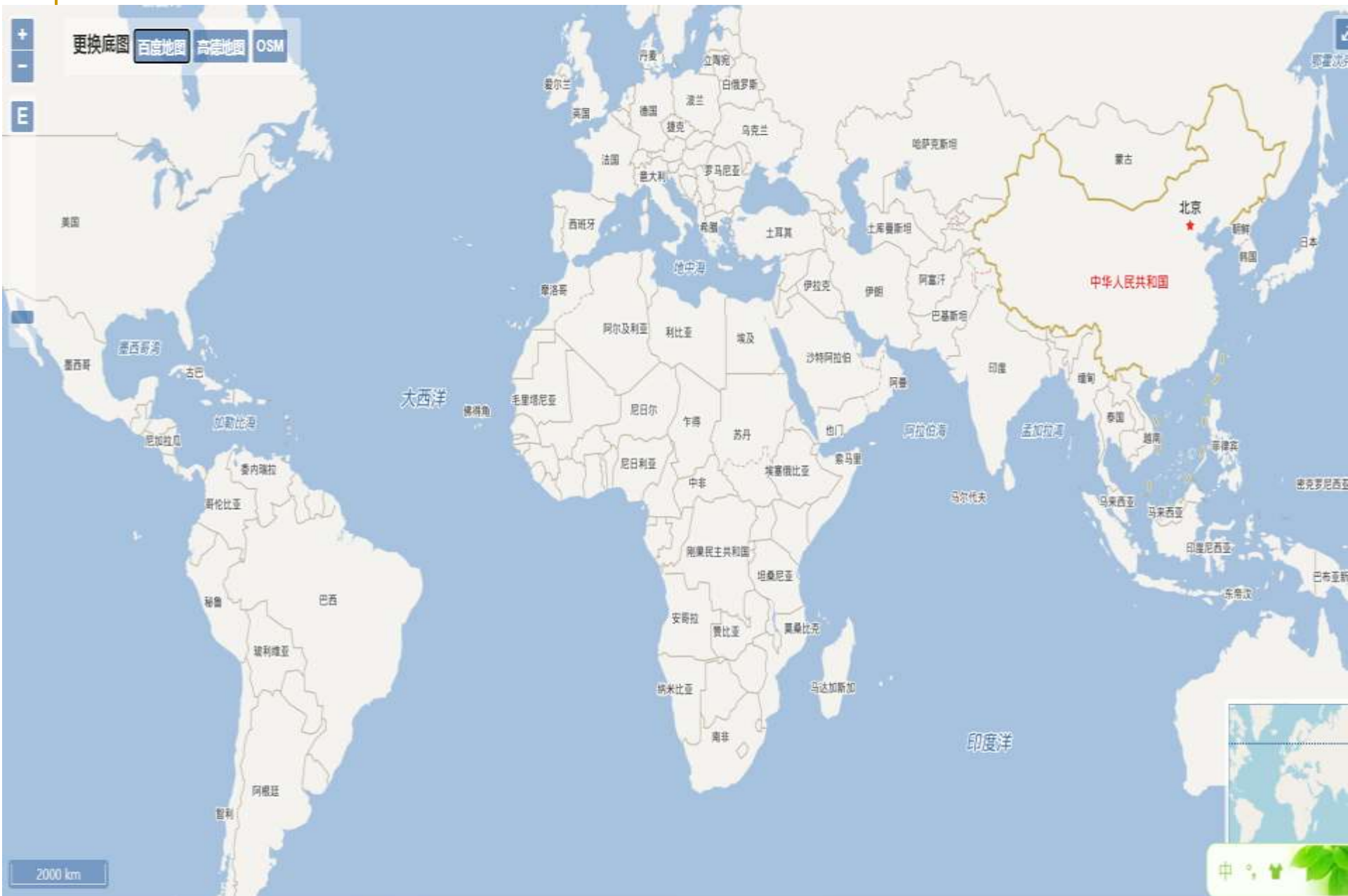
矢量图层通过`ol.layer.Vector`进行创建，`ol.layer.Vector`类包括两个参数：数据源`source`和样式`style`。图层样式使用`ol.style.Style`定义。在样式设置中，`stroke`定义了线的颜色。`ol.style.Style`还有很多的用途，如通过`image`定义点的样式，通过`fill`定义面的填充样式等。

```
const vecLayer = new ol.layer.Vector({  
  source: source1,  
  style: new ol.style.Style({  
    stroke: new ol.style.Stroke({  
      color: '#ff0000',  
      width: 2)  
    })  
  })  
})
```

c.添加图层

使用`map.addLayers(vecLayer)`方法，将所创建的图层添加到地图上。其中`vecLayer`为上一步创建的矢量图层。

(2) 公共地图数据加载



i> 坐标系及投影

- OpenLayers定义了两种常用的投影坐标系，即地理经纬度的EPSG:4326和Web墨卡托的EPSG:3857。OpenLayers 5在地图视图对象中有一个投影坐标系参数(projection)，在加载地图时可通过此参数设置地图的投影坐标系；

`projection:ol.proj.get(EPSG:3857);`

- 在实际应用中，叠加图层数据需要确保在统一的投影坐标系下，OpenLayers 5针对地图投影坐标系的操作，如地图投影转换等，均由`ol.proj.Projection`类实现。

EPSG（**E**uropean **P**etroleum **S**urvey **G**roup）：负责维护全球坐标系统和地理空间数据标准的组织。

ii> 多源数据叠加

在Web端的地图应用中，多源数据叠加显示是一个常用功能。在地图应用中，通常会根据应用的需求来叠加各种类型的数据，并针对叠加的数据进行操作与功能分析。

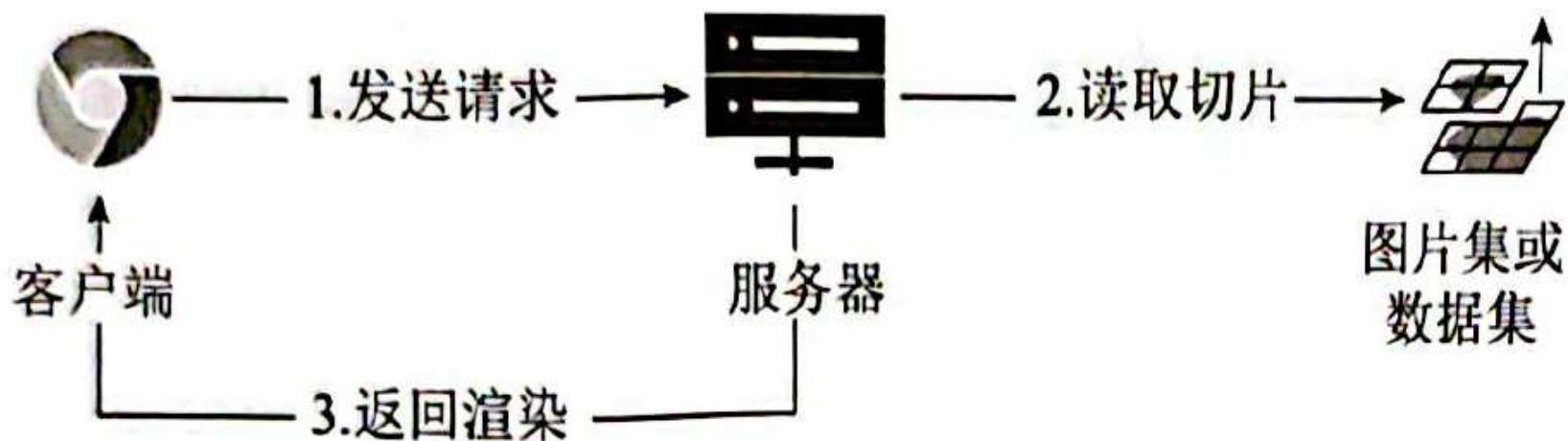
OpenLayers 5支持多源异构数据在同一个地图容器中叠加显示，数据叠加也是OpenLayers 5的地图表现的实现机制。数据叠加非常简单，根据应用需求按照一定的先后顺序添加对应的图层对象到地图容器中即可，但前提是这些多源数据都在同一个投影坐标系下，否则叠加数据将无法吻合。

iii> 地图切片

- 地图切片是采用预生成的方法，将地图存放在服务器端，然后根据用户提交的不同请求，将相应的地图瓦片发送给客户端的过程；
- 它是一种多分辨率的层次模型，按照一定的规则将数据处理成以“级-行-列”方式组织的图片集或数据集，以类似于金字塔的形式存储。从瓦片金字塔底层到顶层，表示的地理范围不变，但分辨率越来越低；
- 地图切片的优势在于能够减小地图加载的时间和带宽消耗，提高地图的显示效率和用户体验。

获取地图切片的流程：

地图切片的加载过程： 客户端提交一个特定地图范围和级别
的请求，服务端通过“级-行-列”，返回对应的底图瓦片给
客户端，进行渲染。



切片服务：

- 常见的切片服务有XYZ、TMS和WMTS等格式；
- XYZ是最常用、也最常见的一种切片服务格式，谷歌、高德、OSM等都是采用这种切片服务；
- XYZ服务在请求时需要在URL中指定xyz参数，其中z为地图级别，x为行号，y为列号。它的切片坐标是EPSG:3857，切片四至为[-20037508.34, -20037508.34, 20037508.34, 20037508.34]。

(3) 几何图形绘制

交互式图形绘制的原理：

- 先初始化一个矢量地图对象并添加到地图容器；
- 然后加载交互式图形绘制控件（在实例化时设置绘制类型，如点、线、规则多边形、任意多边形，以及几何图形对应的特征参数）；
- 最后通过交互式图形绘制控件在地图上绘制相应的几何图形，也可以通过交互式编辑控件来修改已经绘制好的几何图形。

其中，在绘制几何图形时，可以根据需求设置不同的图形样式。在OpenLayers 5中，矢量地图为`ol.layer.Vector`，交互式图形绘制控件为`ol.interaction.Draw`(包括绘制点、线、多边形等)，交互式编辑控件为`ol.interaction.Modify`。另外，在进行编辑时，通常需要先选择某个目标，OpenLayers5提供的选择要素控件为`ol.interaction.Select`。

示例：几何图形绘制

说明：OpenLayers 5提供了用户绘图类`ol.interaction.Draw`，支持绘制点、线、多边形、圆等绘制。例子中用鼠标左键单击地图，地图上就会出现待绘制线的起点，随后移动鼠标，就看到了线，再单击地图，就在这个线上添加了点，依次可以添加多个点，如果要完成线的绘制，在最后一个点处，再点击一下鼠标左键即可。

事件机制：

OpenLayers 5的事件语法统一变更为ES6规范。

- **地图事件类： `ol.MapEvent`**

地图事件的基类，提供了内部的事件类型；

- **地图浏览器事件类： `ol.MapBrowserEvent`**

控制整个地图交互，提供外部设备（如鼠标、触摸屏）与地图交互的主要事件类型，如单击、双击、指针移动、指针拖拽等事件；

- **对象事件类： `ol.Object.Event`** 属性、参数变更事件

包含 `change`、`propertychange` 事件

- **选择控件事件类(`ol.interaction.Select.Event`):** 包含 `select` 事件；

- **绘制控件事件类(`ol.interaction.Draw.Event`):** 包含 `drawstart`、`drawend` 事件；

- **修改控件事件类(`ol.interaction.Modify.Event`):** 包含 `modifystart`、`modifyend` 事件；

- **集合事件类(`ol.Collection.Event`):** 包含 `add`、`remove` 事件；

模块式加载：

<https://www.npmjs.com/package/ol>

第三节 GeoServer

1. GeoServer-介绍
2. GeoServer-Java环境配置与安装
3. GeoServer-发布WMS服务

1. GeoServer-介绍

- 是 OpenGIS Web Service规范的 J2EE 实现，利用 GeoServer 可以方便的发布地图数据，允许用户进行更新、删除、插入数据操作，通过 GeoServer 可以在用户之间迅速共享空间地理信息；
- 兼容 WMS 和 WFS 特性；支持 PostgreSQL、MySQL、Oracle、Shapefile、ArcSDE；支持上百种投影；能够将网络地图输出为 jpeg、gif、png、SVG、KML 等格式；
- 能够运行在任何基于 J2EE/Servlet 容器之上；
- 除此之外还包括许多其他的特性；
- 官方网站：<http://geoserver.org/>

2. GeoServer-Java环境配置与安装

3. GeoServer-发布WMS/WFS服务

参见《指导书》

上机实验4: OpenLayers+GeoServer开发

